

Introduction to the Standard Template Library (STL)

CS101 Introduction to Intermediate Programming CPU Spring 2026

Table of Contents

- [1. Objectives](#)
- [2. Introduction to the Standard Template Library](#)
- [3. Containers](#)
- [4. Sequence containers in C++](#)
- [5. Container adapter classes](#)
- [6. Associative containers in C++](#)
- [7. STL header files](#)
- [8. How can you find out more?](#)
- [9. Iterators](#)
- [10. Types of iterators](#)
- [11. Why Are Iterators Important?](#)
- [12. Algorithms](#)
- [13. Glossary](#)

1. Objectives

- [] Introduction to the Standard Template Library
- [] Sequence containers (like vector)
- [] Container adapter classes (like stack)
- [] Associative containers (like map)
- [] STL header files overview (like <vector>)
- [] Introduction to Iterators (container pointers)
- [] Iterator definition with auto
- [] Reverse iterator and iterator types
- [] Introduction to <algorithm> (like sort)
- [] string as a container-like class

2. Introduction to the Standard Template Library

- The Standard Template Library (STL) is a collection of templates for useful data structures and algorithms.
- It consists of a **runtime** library `std`, which you use as soon as you print anything, and generic templates for classes and functions.

```
#include <iostream>

int main()
{
    std::cout << "I come to you from the STL!\n";
    return 0;
}
```

I come to you from the STL!

- The STL can be grouped in three categories:
 1. **Containers** for creating collections
 2. **Iterators** for accessing collections
 3. **Algorithms** for operating on collections

3. Containers

- **Containers** contain objects that store and organize data.
- Example: The vector class for expandable arrays

```
vector<int> vec {0};           // integer vector with one zero element
cout << vec.size() << endl;   // vector length
vec.push_back(100);           // add value 100 at end of vector
for (int item : vec)          // range-based loop over vector
    cout << item << " ";      // print vector item
```

```
1
0 100
```

- There are two types of STL container classes: **sequence** and **associative**:
 1. **Sequence containers** store data sequentially in memory (like a C-style array) so you have to use indices to access data.
 2. **Container adapter classes** adapt one of the other container to a specific use (like FIFO or LIFO).
 3. **Associative containers** store data nonsequentially so that elements can be located faster (by using a key).
- Containers are C++ "template" implementations of important data structures. We'll cover five of these, and two in great detail.

[X] **vector**

dynamic array, fast random access

[X] **map**

sorted key-value pairs (balanced BST, $O(\log n)$)

[] **unordered_map**

hash table, average $O(1)$ access

[] **set**

sorted unique values (like map with just keys)

[] **unordered_set**

hash set

[X] **list**

doubly linked list, fast insertion/deletion anywhere

[X] **queue**

FIFO structure (adapter over deque)

[] **deque**

double-ended queue, fast insertion/removal at both ends

[] **priority_queue**

max-heap structure (adapter over vector)

[X] **stack**

LIFO structure (adapter over deque)

4. Sequence containers in C++

We're going to cover all of these.

CLASS	DESCRIPTION
array	Fixed-size container similar to a C-style array
vector	An expandable array that adjusts automatically
forward_list	A singly linked list of data elements
list	A doubly linked list of data elements
deque	A double-ended queue (similar to a vector)

Why is string not in this list?

- string is a sequence class specialized for characters with its own header <string> and interface.
- It does support iterators, begin() and end()
- It does support size(), empty(), [] and at()
- It works with STL algorithms like sort

String examples:

```
string name = "Marcus Birkenkrahe";  
  
for (char c : name) // uses iterator (compiler rewrites it)  
    cout << c;  
  
cout << "\nChars = " << name.size(); // size()  
  
cout << "\nFifth char = " << name.at(4) << " " << name[4];
```

```
Marcus Birkenkrahe  
Chars = 18  
Fifth char = u u
```

The range-based loop is "syntactic sugar". The compiler rewrites it:

```
for (auto it=name.begin(); // start  
     it != name.end();     // stop  
     ++it) {               // move  
    char c = *it;  
    cout << c;  
}
```

5. Container adapter classes

Container adapter classes are not themselves containers. Instead, they are classes that adapt one of the containers to a specific purpose:

CLASS	DESCRIPTION
stack	Last-in, first-out (LIFO) container using a deque, always gives you the last element that was inserted
queue	First-in, first-out (FIFO) container using a deque always gives you the first/earliest element inserted
priority_queue	Always gives you the element with the greatest value, elements are by default stored in a vector

6. Associative containers in C++

CLASS	DESCRIPTION
set	Sorted set of unique values without duplicates
unordered_set	Like a set but elements are not sorted
multiset	set of values with duplicates allowed
unordered_multiset	A multiset but elements are not sorted
map	Maps a set of keys to unique key-ordered data elements
unordered_map	Like a map but elements are not sorted
multimap	Many keys per data element, duplicates allowed
unordered_multimap	Like a multimap but elements are not sorted

All of these associative containers get their element association from the fact that they are implemented as **binary search trees** (more specifically, **self-balanced, red-black binary search trees**).

7. STL header files

To work with these containers, you need to include the necessary header files:

HEADER FILE	CLASSES
<array>	array (this is not the C-style array)
<deque>	deque
<forward_list>	forward_list
<list>	list
<map>	map, multimap
<queue>	queue, priority_queue
<set>	set, multiset
<stack>	stack

HEADER FILE	CLASSES
<unordered_map>	unordered_map, unordered_multimap
<vector>	vector

I have ever only worked with half of these myself.

8. How can you find out more?

cppreference.com

Page
Discussion

Planned Maintenance
The site will be in a temporary read-only mode in the next few weeks to facilitate some long-overdue software updates. We apologize for any inconvenience this may cause!

C++ reference

C++11, C++14, C++17, C++20, C++23, C++26 | Compiler support C++11, C++14, C++17, C++20, C++23, C++26

Language Preprocessor – Comments ASCII chart Basic concepts Keywords Names (lookup) Types (fundamental types) The main function Modules (C++20) Contracts (C++26) Expressions Value categories Evaluation order Operators (precedence) Conversions – Literals Constant expressions Statements If – switch for – range-for (C++11) while – do-while Declarations – Initialization Functions – Overloading Coroutines (C++20) Classes (unions) Templates – Exceptions Freestanding implementations Standard library (headers) Named requirements Language support library Program utilities Signals – Non-local jumps Basic memory management Variadic functions source_location (C++20) Comparison utilities (C++20) Type support – type_info numeric_limits – exception initializer_list (C++11) Coroutine support (C++20) Contract support (C++26) Concepts library (C++20)	Diagnostics library Assertions – System error (C++11) Exception types – Error numbers basic_stacktrace (C++23) Debugging support (C++26) Memory management library Allocators – Smart pointers Memory resources (C++17) Metaprogramming library (C++11) Type traits – ratio integer_sequence (C++14) General utilities library Function objects – hash (C++11) Swap – Type operations (C++11) Integer comparison (C++20) pair – tuple (C++11) optional (C++17) expected (C++23) variant (C++17) – any (C++17) bitset – Bit manipulation (C++20) Containers library vector – deque – array (C++11) list – forward_list (C++11) inplace_vector (C++26) hive (C++26) map – multimap – set – multiset unordered_map (C++11) unordered_multimap (C++11) unordered_set (C++11) unordered_multiset (C++11) Container adaptors span (C++20) – rdsan (C++23) Iterators library Ranges library (C++20) Range factories – Range adaptors generator (C++23) Algorithms library Numeric algorithms Execution policies (C++17) Constrained algorithms (C++20)	Strings library basic_string – char_traits basic_string_view (C++17) Text processing library Primitive numeric conversions (C++17) Formatting (C++20) – Localization text_encoding (C++26) Regular expressions (C++11) basic_regex – Algorithms Default regular expression grammar Null-terminated sequence utilities: byte – multibyte – wide Numerics library Common math functions Mathematical special functions (C++17) Mathematical constants (C++20) Basic linear algebra algorithms (C++26) Data-parallel types (SIMD) (C++26) Pseudo-random number generation Floating-point environment (C++11) complex – valarray Date and time library Calendar (C++20) – Time zone (C++20) Input/output library Print functions (C++23) Stream-based I/O – I/O manipulators basic_istream – basic_ostream Synchronized output (C++20) File systems (C++17) Concurrency support library (C++11) thread – jthread (C++20) atomic – atomic_flag atomic_ref (C++20) – memory_order Mutual exclusion – Condition variables Futures – Semaphores (C++20) latch (C++20) – barrier (C++20) Safe Reclamation (C++26) Execution support library (C++26) Feature test macros (C++20) Language – Standard library – Headers
---	--	---

Technical specifications
Standard library extensions (library fundamentals TS)
resource_adaptor – invocation_type
Standard library extensions v2 (library fundamentals TS v2)
propagate_const – ostream_joiner – randint
observer_ptr – Detection idiom
Standard library extensions v3 (library fundamentals TS v3)
scope_exit – scope_fail – scope_success – unique_resource

Parallelism library extensions v2 (parallelism TS v2)
simd
Concurrency library extensions (concurrency TS)
Transactional Memory (TM TS)
Reflection (reflection TS)

External Links – Non-ANSI/ISO Libraries – Index – std Symbol Index

Figure 1: Screenshot of cppreference.com

This is especially useful if you want to keep on top of changes to C++ (currently C++23 while we use C++20) in line with GCC 11:

```
gcc --version
```

```
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
```

warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

9. Iterators

- **Iterators** for objects that behave like pointers, and are used to access the individual data elements in a container.
- To use an iterator, you need to

1. Define an iterator variable (for the container type):

```
array<string,3> names = {"Jim", "Joe", "Jane"};  
array<string,3>::iterator it;
```

2. Get an iterator object from the container:

```
it = names.begin(); // point it at the start of names
```

3. Assign the iterator object to the variable:

```
cout << *it; // dereference the iterator to get to the value
```

- Example: STL iterator with vector

```
vector<int> nums {10,20,30,40}; // vector with four nonzero integers  
  
vector<int>::iterator it; // iterator for an integer vector  
for (it=nums.begin(); it != nums.end(); ++it) // use like loop variable  
{  
    cout << *it << " "; // to access value you must dereference  
}
```

```
10 20 30 40
```

- Explanation:
 - `it` is an iterator pointing to elements in the vector.
 - `*it` dereferences to get the vector value.
 - `++it` advances pointer to the next element (pointer arithmetic!)
- String example:

```
string name = "Marcus Birkenkrahe";  
  
std::string::iterator it; // define iterator  
for (it=name.begin(); it != name.end(); ++it)  
    cout << *it;
```

```
Marcus Birkenkrahe
```

- Example using the array class (shown above):

```
#include <iostream>
#include <array>
using namespace std;

int main()
{
    array<string,3> names = {"Joe", "Jim", "Jane"};
    array<string,3>::iterator it;
    it = names.end();
    cout << *(it-1);
    return 0;
}
```

Jane

- In the example, `cout << *it;` leads to undefined behavior because `names.end()` does not point to the last element! So `names.end()` cannot be dereferenced! Not very safe!
- You can use `auto` to simplify the definition:

```
vector<int> nums {10,20,30}; // vector
auto it = nums.begin();      // iterator definition
```

- To reverse the iteration, you cannot just switch to `it--` but you have to use a different `reverse_iterator` with `rbegin()` and `rend()`:

```
vector<int> nums {10, 20, 30, 40};
vector<int>::reverse_iterator it;
for (it = nums.rbegin(); it != nums.rend(); it++)
    cout << *it << " ";
```

40 30 20 10

10. Types of iterators

There are six categories of iterators:

- **Forward** iterator can only move forward in a container (using `++`)
- **Bidirectional** iterator can move forward and backward (uses `++`, `--`)
- **Random access** iterators can move forward, backward, and jump
- **Input** and **output** iterators are used with input/output streams to read/write data.
- **Contiguous** iterators are random-access iterators that point to elements stored in contiguous memory locations.

The type of container you use determines the iterator category:

Container Type	Iterator Category
vector, deque	Random-access iterator

Container Type	Iterator Category
list, set, map	Bidirectional iterator
forward_list, unordered_*	Forward iterator

11. Why Are Iterators Important?

- Iterators generalize the idea of looping over elements. You could loop with `for (int i = 0; i < vec.size(); ++i)` — but this only works for containers that support **indexing** (like vector, array).
- Many STL containers (like list, set, map) do **not** support direct indexing. Iterators provide a uniform way to:
 1. Traverse **any** container
 2. Work with STL algorithms (sort, find, remove_if)
 3. Abstract away the container **type**
 4. Avoid off-by-one and index **errors**
 5. Support bidirectional or random access where possible

Iterators decouple the traversal logic from the container's internal structure.

They let your code be more **generic**, **safer**, and **reusable**.

12. Algorithms

- **Algorithms**, function templates that perform various operations on elements of containers.
- Example: `std::sort` on a vector - the function sorts in place and does not return a value. So nums will be changed in this code.

```
#include <iostream>    // for std::cout
#include <vector>       // for std::vector
#include <algorithm>    // for std::sort
using namespace std;

int main()
{
    vector<int> nums { 42, 5, 17, 8 };

    sort(nums.begin(), nums.end()); // ascending sort "in place"

    for (int n : nums)
        cout << n << " ";

    return 0;
}
```

5 8 17 42

- Example: `std::sort` on a string - the function returns void, so you must copy name first, then sort the copy (if you want to keep name).

```
#include <algorithm>
#include <string>
```



```
#include <iostream>

int main() {
    std::string name = "emily";
    std::string sorted_name = name; // copy first
    std::sort(sorted_name.begin(), sorted_name.end()); // then sort

    std::cout << sorted_name << " (was: " << name << ")\n";

    return 0;
}
```

```
eilmy (was: emily)
```

13. Glossary

TERM	EXPLANATION
STL	Standard Template Library
vector	An expandable array
Container	Data structure for objects that store
Iterator	Data structure used to access containers
Algorithm	Function templates operating on containers
Sequence container	Data are stored sequentially in memory
Associative container	Data are stored non-sequentially in memory
Container adapter class	Class that uses containers
Header files	Contains container constants and functions
string	Container-like sequence class

Author: Marcus Birkenkrahe

Created: 2026-04-25 Sat 08:20